LEAF: Looped Entity Attention Fusion for Population-Generalizable Multi-Agent Defense

Anonymous Authors

***Abstract*—Cooperative target protection requires defenders to interpret threats in the context of nearby teammates and protected assets as their populations change. We propose Looped Entity Attention Fusion (LEAF), a receiver-conditioned relational encoder for cooperative value learning. A shared attention block iteratively updates the entity representations, a query derived from the acting defender reads each intermediate representation, and learned fusion combines these reads before a recurrent decision network. This design couples current-scene contextual processing with intermediate-stage access while retaining temporal memory downstream. In SkyHAD, policies trained with at most ten vehicles per team and three targets are evaluated without retraining on 24 configurations reaching 50 vehicles per team and 12 targets. Across three training seeds, LEAF improves mean reward over REFIL by 0.800, 1.635, and 2.211 in target-count, team-size, and joint extrapolation, respectively. Trained architectural comparisons favor contextual processing before reading and current-state queries with downstream memory. Normal forward records show receiver-specific fusion concentrated on early contextual reads, while evaluations across trained depths characterize scene-dependent refinement. At 50 vehicles per team and two targets, attacker interception increases from 72.27% to 78.68%, while the incidence of target damage decreases from 70.56% to 28.11%. These results support relational encoding as a practical approach to population transfer in cooperative defense.**

Note to Practitioners— A cooperative protection controller may need to operate with different numbers of defending vehicles, incoming threats, and protected assets. LEAF uses a shared policy that accepts a variable-size set of observed entity states. Each defender builds the current scene context, reads it from its own perspective, and combines it with a recurrent temporal state to choose an action. The controller is trained on small engagements and reused with frozen parameters in larger ones. Evaluation measures both protected-asset reward and friendly-collision mortality, allowing the two operational outcomes to be assessed together. The study uses the planar vehicle model in SkyHAD with public entity-state observations and a reactive attacker. This interface suits simulation-based assessment of cooperative protection strategies and can serve as the policy layer when integrating platform-specific sensing and actuation.

***Index Terms*—Multi-agent reinforcement learning, cooperative control, population generalization, entity attention, target protection.**

I. Introduction

COOPERATIVE target protection requires a team to respond to several threats at once. A defender approaching one attacker must account for the target behind that attacker, nearby teammates, and competing threats elsewhere. As vehicles are intercepted and threats move between targets, the useful relationships change. A controller trained for a small engagement should reuse these relationships when more defenders, attackers, or protected assets enter the scene. This requirement motivates policies that coordinate through entity relationships rather than a fixed population layout.

Entity-based multi-agent reinforcement learning provides a suitable starting point. Shared set encoders accept variable numbers of entities, while attention extracts information for each agent [1], [2]. REFIL further encourages reusable interactions through randomized entity-wise value factorization [3]. The remaining design question concerns the representation supplied to the controller: how should an individual defender interpret an attacker together with its surrounding teammates and targets, and retain the parts of that context that matter for its next action?

We address this question through *receiver-conditioned, multi-stage relational encoding*. A shared attention block repeatedly updates the entity representations within one physical decision. Each update operates on the context produced by the preceding update. A reader conditioned on the defender's own state accesses every intermediate representation, and a learned fusion combines these reads into the decision feature. Context construction and reading form one encoder: the loop develops relations among entities, while the reader expresses those relations from the perspective of the acting defender. Keeping intermediate reads lets the controller use early context alongside later refinements.

This computation has a different role from memory across physical time. An attacker's target assignment is private, although its position and velocity are observed. The policy therefore retains a recurrent state to interpret motion over successive observations. We use the current self state to query the relational encoder and place temporal recurrence downstream. The resulting architecture, **Looped Entity Attention Fusion (LEAF)**, augments REFIL's feature with the fused relational read and then updates the defender's GRU once. Network parameters are shared across entities, defenders, and processing rounds. Training samples one to four rounds, and the deployed policy uses four.

Our evaluation follows the intended control problem. In SkyHAD [4], policies learn with at most ten vehicles per team and three targets, then execute with frozen parameters on 24 configurations reaching 50 vehicles per team and 12 targets. We compare LEAF with REFIL, QMIX-Atten, DCG, SPECTra, ALMA, and a rule-based controller. Reward is the negative of accumulated target damage. LEAF attains the highest group mean reward among these methods in the in-

distribution, target-count, team-size, and joint-extrapolation groups. Its gains over REFIL are accompanied by fewer friendly-collision deaths and more attacker interceptions.

The experiments connect this outcome to the representation design. Trained comparisons examine reading before and after contextual processing, placing memory in the reader, and retaining the REFIL host. Execution at the four depths used in training characterizes how refinement affects the frozen policy. Normal forward records then show which intermediate reads supply its decisions, while common-scene trajectories connect interception behavior to target protection. The contributions are:

1) A receiver-conditioned relational encoder that couples shared iterative entity updates with fusion of intermediate reads, integrated with recurrent cooperative value learning.
2) A population-transfer evaluation covering changes in vehicle and target counts, with learning curves, two operational metrics, and trained architectural comparisons across three training seeds.
3) An empirical account of the controller's operation that connects contextual processing and intermediate-read use to interception and target-protection outcomes.

II. Related Work

A. Target Defense and Reach–Avoid Games

Cooperative target defense couples interception with protection: defenders must account for both approaching adversaries and the assets they threaten. Differential-game approaches characterize this interaction through geometry, winning regions, and feedback strategies. Garcia et al. derive state-feedback optimal strategies for active defense [5]. For larger teams, Chen et al. combine pairwise reach–avoid outcomes through matching [6], while Yan et al. use analytical barriers to determine coalition capabilities and formulate task assignment in convex domains [7]. Matching-based strategies also address three-dimensional heterogeneous teams, combining capture subgames with repeated allocation [8].

Perimeter defense emphasizes the team outcome when multiple intruders approach a protected region. Shishika et al. develop a decomposition and assignment strategy that explicitly accommodates cooperative defense [9]. This literature establishes that interception opportunities depend on the joint arrangement of defenders and intruders, making the distribution of team effort central to protection. The analytical methods obtain guarantees under their specified dynamics and geometry. Our setting evaluates a learned maneuver policy using cumulative damage to stationary targets, with the same policy deployed across different vehicle and target populations. These changes alter the interactions that the policy must represent and the allocation of effort implicit in its actions.

B. Task Allocation and Cooperative Learning

Task allocation provides an explicit way to organize multi-robot effort. Gerkey and Matarić give a formal taxonomy of allocation problems [10], and consensus-based decentralized auctions combine local bidding with conflict resolution [11]. Cooperative tasks can require robot coalitions: Deng et al. develop distributed auctions for two-robot coalitions, including time-varying assignments and multiplayer reach–avoid applications [12]. Miele et al. integrate distributed allocation with safe movement, connectivity maintenance, and deadlock handling [13]. These approaches expose assignment and motion coordination as coupled decisions.

Learning offers complementary ways to couple these decisions. Centralized training with decentralized execution supports cooperation through multi-agent actor–critic methods and team-reward policy optimization [14], [15]. Coach-player learning additionally addresses changing team composition through individual strategies supplied by a coordinating coach [16]. Cooperative distributional reinforcement learning has been applied to dynamic multi-ship weapon–target assignment with attention and delayed rewards [17]. In automation settings, hierarchical learning combines allocation and planning for uncertain swarm confrontation [18], and observer-assisted multi-agent learning addresses pursuit–evasion under unknown uncertainties [19]. Closely related to target protection, Li et al. organize learning into intention recognition, target allocation, and path execution [20]. Their hierarchy uses differential-game information during allocation training. LEAF instead learns defender action values through a common entity representation, allowing coordination to emerge through maneuver decisions without a separate assignment output. Its contribution concerns the joint relational encoder that supplies these values; the cooperative value-learning objective remains based on REFIL.

C. Entity Representation and Multi-Stage Relational Encoding

Variable-size entity representations are established tools for cooperative learning. Deep Sets and Set Transformers encode unordered inputs [1], [2], while swarm reinforcement learning uses mean embeddings to represent changing neighborhoods [21]. Value decomposition, monotonic mixing, and coordination graphs connect agent utilities with team objectives [22]–[24]. REFIL combines entity attention with randomized entity partitions and an auxiliary factorized value objective [3]. UPDeT and TransfQMix use transformers in cooperative policies and mixing networks [25], [26]; SPECTra emphasizes permutation-consistent scalable networks [27], and ALMA addresses composite tasks through hierarchical learning [28]. Spatial aggregation and temporal recurrence have also been combined for perimeter defense: GR-MAPPO uses GraphSAGE and a GRU and evaluates cross-scale transfer [29].

Relational reinforcement learning models entity interactions with attention [30]. Universal Transformers reuse a transition across depth [31], Jumping Knowledge networks aggregate intermediate graph representations [32], and LayerDrop trains varying-depth computation paths [33]. Attention-based recurrence addresses temporal information in multi-agent learning [34]. Building on these precedents, LEAF treats shared contextual updates, self-conditioned intermediate reading, and cross-round fusion as one receiver-conditioned

TABLE I
SKYHAD PLANAR TARGET-PROTECTION INTERFACE.

Quantity	Setting
Arena	5000×5000 m
Decision interval / horizon	1 s / 100 steps
Acceleration limit	$40\,\mathrm{m/s^2}$
Vehicle speed range	35–120 m/s
Swept collision distance	20 m
Automatic firing separation	Below 200 m, pre-movement
Damage profile	Unit core to 200 m; linear to 0 at 400 m

multi-stage relational encoder. One current-self query reads each intermediate entity set, and learned fusion supplies a residual feature to the host's recurrent utility network. The retained GRU carries information across physical decisions. This organization separates contextual processing at a decision from temporal memory across decisions, while jointly learning the loop and its readout for cooperative protection.

III. Problem Formulation and Preliminaries

A. Cooperative Target Protection

We study cooperative target protection in the planar particle model of SkyHAD 3.0.0 [4] in which defenders coordinate their actions against incoming attackers across a changing set of protected targets. Let $\mathscr{R}$, $\mathscr{B}$, and $\mathscr{T}$ denote the defender, attacker, and target sets, with initial configuration $\mathbf{n} = (N_R, N_B, K)$. Targets occupy the left portion of the arena; attackers approach from the right, and defenders distribute their responses between interception and target coverage. Population transfer changes the sizes of these sets while preserving the interaction rules.

Vehicles choose among eight normalized acceleration directions and zero acceleration. Close vehicle encounters can remove both participants, while automatic firing allows defenders to engage attackers and attackers to damage targets. Targets remain as damage accumulates. The simulator continues after the final defender disappears and ends when all attackers have disappeared or the episode horizon is reached. Table I collects the physical interface used by all policies.

Public observations contain entity positions, velocities, types, health or alive status, and target damage. The fixed reactive attacker controller selects a target using public defender coverage and distances, then carries out a rush and periodically revises its assignment. That assignment is private. Recent motion therefore provides useful context for interpreting the attackers' evolving objectives.

Let $\Delta d_{k,t}$ be damage to target k at step t. The physical step reward and unshaped episode reward are

$$\Delta D_t = \sum_{k \in \mathscr{T}} \Delta d_{k,t}, \qquad r_t = -\Delta D_t, \qquad R = \sum_t r_t = -D. \quad (1)$$

Here $D = \sum_t \Delta D_t$. Higher episode reward R indicates better protection, and $R = 0$ denotes no target damage. Comparisons use identical configurations and initial-scene conditions.

B. Population Generalization

With the attacker controller π_B fixed, the team faces a family of cooperative partially observable decision processes indexed by $\mathbf{n}$. Defender i acts from its observation–action history $\tau_{i,t}$ through a shared policy π_θ, maintaining its own temporal state. The latent state includes the private attacker assignment.

Training samples small engagements. Evaluation freezes θ and varies vehicle populations, target populations, or both, while retaining the physical rules, action semantics, and attacker controller. The quantity of interest is $\mathbb{E}[R \mid \mathbf{n}, \pi_\theta, \pi_B]$ across evaluation configurations. This formulation connects population transfer to a concrete control requirement: distributing defensive responses among a changing set of interacting entities.

C. Entity-Factorized Cooperative Learning

LEAF builds on REFIL's entity-based utility network and imagined decomposition [3]. A shared encoder and masked attention produce a base feature $\ell_{i,t}$ for each defender. A recurrent utility network combines that feature with individual memory to produce $Q_i(\tau_{i,t}, a_{i,t})$. Centralized training uses a scene-conditioned monotonic mixer,

$$Q_{\mathrm{tot}}(x_t, a_t) = f_{\mathrm{mix}}(Q_1, \ldots, Q_{N_R}; x_t), \quad \frac{\partial Q_{\mathrm{tot}}}{\partial Q_i} \geq 0, \quad (2)$$

so individual greedy actions are consistent with a maximizer of the represented joint value over legal action sets [23].

REFIL draws random entity partitions and forms in-group and out-group masked views of each recorded sequence. These views produce an auxiliary team value that shares the main branch's temporal-difference target. LEAF retains this learning interface and augments the feature entering the recurrent utility network. Each branch applies its entity mask and reconstructs its temporal history with shared parameters. Deployment uses the main branch.

IV. Looped Entity Attention Fusion

LEAF constructs a decision feature in two steps: it develops context among the observed entities and reads that context for the acting defender. A shared loop and an intermediate-read fusion implement these steps jointly. The fused feature enters a recurrent utility network, which maintains the defender's history across physical decisions. Figure 1 shows the information flow.

A. Receiver-Relative Entity Encoding

At physical time t, defender i receives the entity matrix $X_{i,t}$. Other entities are expressed relative to this defender, while the self row retains its absolute features. Inputs include normalized positions and velocities, entity type, health or alive status, target damage, and previous actions. Let $m_{i,t,e} \in \{0,1\}$ indicate whether entity e is a permitted key. The main branch uses the observed entities; REFIL's imagined branches additionally apply their partition masks.

A shared encoder produces entity embeddings $e_{i,t,e}$ and the REFIL feature $\ell_{i,t}$. The initial relational representation has width $d = 64$:

$$H^{(0)}_{i,t,e} = W_g e_{i,t,e} + b_g. \quad (3)$$

The self token supplies the receiver's query, and the mask determines the entities participating in contextual updates

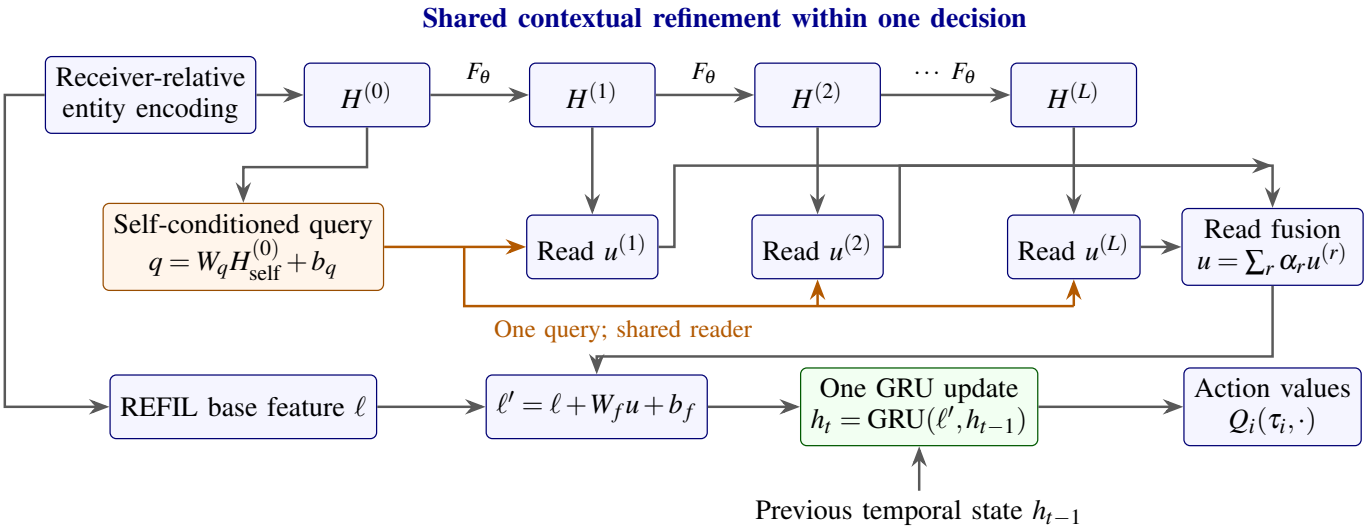


Fig. 1. LEAF at one defender decision. A shared contextual-update block processes the receiver-relative entities for L rounds. The defender's self-state query reads each intermediate representation, and learned fusion combines the resulting summaries. The fused read augments the REFIL feature before one GRU update. The loop processes the current observation; the GRU carries memory between observations. Training samples $L \in \{1,2,3,4\}$ and formal evaluation uses $L = 4$.

and reading. All defenders use the same parameters. A new observation initializes a new entity representation, so changing population size changes the input set while preserving the encoder and action interface.

B. Shared Refinement and Intermediate Reading

Suppressing defender and time indices, one contextual update is

$$\begin{aligned} J^{(r)} &= \text{LN}_a(H^{(r-1)}), \\ \widetilde{H}^{(r)} &= H^{(r-1)} + \text{MHA}_\theta(J^{(r)}, H^{(r-1)}, H^{(r-1)}; m), \\ H^{(r)} &= \text{Mask}_m\left[\widetilde{H}^{(r)} + \text{FFN}_\theta(\text{LN}_f(\widetilde{H}^{(r)}))\right]. \end{aligned} \tag{4}$$

Here LN is rowwise layer normalization, $\text{MHA}(Q,K,V;m)$ is masked multihead attention, and Mask_m zeros excluded rows. The feedforward layer has hidden width $2d$. The same parameters θ are used for all L rounds. Every permitted entity can interact in the first round; subsequent rounds update features that already contain entity context. For example, an attacker's representation can be processed together with representations of the defenders and targets around it. The shared operator learns this contextual transformation across processing depths [30], [31].

The acting defender reads each intermediate representation with one query derived from its initial self token:

$$\begin{aligned} q_{i,t} &= W_q H^{(0)}_{i,t,\text{self}} + b_q, \\ u^{(r)}_{i,t} &= \text{MHA}_{\text{read}}(q_{i,t}, H^{(r)}_{i,t}, H^{(r)}_{i,t}; m_{i,t}). \end{aligned} \tag{5}$$

The query and reader parameters are shared across rounds at a decision. This produces a sequence of defender-specific summaries $u^{(1)}_{i,t}, \ldots, u^{(L)}_{i,t}$. The loop supplies the entity context; the query determines how the receiver accesses it.

A scalar scoring map combines these summaries:

$$\alpha^{(r)}_{i,t} = \frac{\exp(w_s^\top u^{(r)}_{i,t} + b_s)}{\sum_{j=1}^{L} \exp(w_s^\top u^{(j)}_{i,t} + b_s)}, \qquad u_{i,t} = \sum_{r=1}^{L} \alpha^{(r)}_{i,t} u^{(r)}_{i,t}. \tag{6}$$

Entity attention selects information within a processing stage; fusion combines information across stages. The weights are computed from the receiver-specific reads at each decision. Preserving these reads gives the controller direct access to intermediate context [32]. All L rounds are executed, and fusion determines their contribution to the final feature.

C. Recurrent Cooperative Decision

The relational read augments the REFIL feature through a residual projection:

$$\begin{aligned} \ell'_{i,t} &= \ell_{i,t} + W_f u_{i,t} + b_f, \\ h_{i,t} &= \text{GRU}(\ell'_{i,t}, h_{i,t-1}), \\ Q_i(\tau_{i,t}, \cdot) &= W_o h_{i,t} + b_o. \end{aligned} \tag{7}$$

The residual projection is initialized to zero; both feature paths are then learned jointly. The GRU updates once per physical decision and carries its hidden state to the next observation. Thus processing round r develops current entity context, while physical time t advances the temporal state. This placement lets the reader use the current defender state and lets the decision network combine that read with its motion history. Decentralized execution selects the legal action with highest utility.

The trained memory-query comparison adds the preceding GRU state to Eq. (5), with the gradient stopped through that query input. Both designs retain the recurrent decision network. The comparison therefore examines the location of temporal information within the encoder.

Algorithm 1 LEAF at one defender decision

Require: Entities $X_{i,t}$, mask $m_{i,t}$, memory $h_{i,t-1}$, depth L
Encode entities and compute base feature $\ell_{i,t}$
Initialize $H_{i,t}^{(0)}$ and query $q_{i,t}$
for $r = 1, \dots, L$ **do**
    Update $H_{i,t}^{(r)}$ with shared block, Eq. (4)
    Read $u_{i,t}^{(r)}$ with $q_{i,t}$, Eq. (5)
end for
Fuse reads and augment $\ell_{i,t}$, Eqs. (6)–(7)
Update temporal state and compute action values
return $\arg\max_{a \in \mathscr{U}_{i,t}} Q_i(\tau_{i,t}, a)$ and $h_{i,t}$

D. Training and Population Transfer

Collection samples L uniformly from $\{1,2,3,4\}$ and holds it fixed for an episode. Replay independently samples one depth per minibatch for the online main and imagined branches and the target network. The shared block is consequently trained both as a short contextual transform and as an operator on its own preceding outputs. Formal evaluation fixes $L = 4$.

Training retains the host's potential-shaped reward,

$$\widetilde{r}_t = -\Delta D_t + \gamma \Phi(s_{t+1}) - \Phi(s_t),$$
$$\Phi(s) = -\sum_{j \in \mathscr{B}_{\text{alive}}} \text{clip}\left(1 - \frac{\min_{k \in \mathscr{T}} \|p_j - p_k\|}{4000}, 0, 1\right), \quad (8)$$

where p_j and p_k are attacker and target positions and $\gamma = 0.99$. This potential provides a dense proximity signal. Evaluation uses the unshaped reward $R = -D$.

Let $\bar{r}_t$ denote the replay reward and b_t the bootstrap multiplier. Natural termination uses $b_t = 0$; ordinary transitions use $b_t = \gamma$, and time-limit transitions retain a successor. If the final defender disappears, the discounted reward tail from the remaining simulation is included in its final decision transition. Online utilities select the legal greedy successor actions a_{t+1}^*, giving the target

$$y_t = \bar{r}_t + b_t Q_{\text{tot}}^-(x_{t+1}, a_{t+1}^*). \quad (9)$$

The main and imagined values are trained against this target:

$$\mathscr{L} = (1-\lambda)\mathscr{L}_{\text{main}} + \lambda \mathscr{L}_{\text{aux}}, \qquad \lambda = 0.5,$$
$$\mathscr{L}_k = \frac{\sum_{b,t} v_{b,t} (Q_{k,b,t} - y_{b,t})^2}{\sum_{b,t} v_{b,t}}, \quad k \in \{\text{main}, \text{aux}\}, \quad (10)$$

where $v_{b,t}$ marks valid replay transitions and the target is detached during optimization. Both branches share parameters and reconstruct their masked histories. Deployment uses the main branch, as summarized in Algorithm 1.

Shared rowwise maps and masked attention make the encoder consistent with entity ordering: permuting non-self entities and their masks permutes the intermediate representations and preserves the receiver's read and action values. Parameter count is independent of entity count and unfolding depth. For M entity slots and width d, the relational updates cost $O(L(M^2 d + M d^2))$ per receiver. The same controller can therefore accept different populations through its entity set and mask; Section V evaluates the resulting protection performance.

V. Experimental Evaluation

Setup. We evaluate target protection in the planar model of SkyHAD 3.0.0 [4], with public entity-state observations, a fixed reactive–rush attacker, and a horizon of 100 physical steps. Training samples balanced teams $N_R = N_B = N \in \{4,6,8,10\}$ and target counts $K \in \{1,2,3\}$. Each learned method uses three training seeds and a budget of 10^6 physical environment steps per seed. LEAF has hidden width 64, four attention heads, learning rate 5×10^{-4}, discount 0.99, batch size 32, and replay capacity 5000. Training samples one to four relational rounds; formal evaluation uses four.

Final policies execute with frozen parameters on 24 configurations, each with 300 common initial scenes. In-distribution (ID) configurations are $(4,4,2)$, $(6,6,2)$, $(8,8,2)$, and $(10,10,3)$, written as (N_R, N_B, K). Target-count extrapolation (K-OOD) uses $(10,10,K)$ for $K \in \{4,6,9,12\}$; team-size extrapolation (N-OOD) uses $(N,N,2)$ for $N \in \{15,20,25,30,40,50\}$. Joint extrapolation combines $N \in \{15,20,30\}$ with $K \in \{4,6\}$ and includes $(30,30,9)$ and $(30,30,12)$. The remaining configurations, $(5,5,2)$ and $(10,10,2)$, provide intermediate-scale reference points. The five learned baselines are REFIL [3], QMIX with an entity-attention encoder (QMIX-Atten) [23], DCG [24], SPECTra [27], and ALMA [28]. Rule provides a fixed-controller reference.

Metrics and aggregation. The primary metric is episode reward $R = -D$, with higher values indicating better target protection. Friendly-collision mortality,

$$C = \frac{\text{defenders killed by friendly collision}}{N_R}, \quad (11)$$

measures losses within the protecting team. Each model first averages the 300 scenes within a configuration and then averages configurations equally within a group. Tables report mean $\pm$ sample standard deviation (SD) across the three trained models [35]. For Rule, the reported SD is across 300 scene seeds after averaging configurations within each seed.

Learning curves use 50 validation checkpoints. Each checkpoint evaluates four ID configurations with 25 fixed scenes each, separate from the formal evaluation scenes. We align validation indices across training seeds and plot the mean reward at the mean recorded environment-step count. Bands show the three-model sample SD, without smoothing.

A. Comparison Across Deployment Populations

LEAF obtains the highest group mean reward among the six learned methods and Rule in all four groups (Table II). Relative to REFIL, the reward gains are 0.310, 0.800, 1.635, and 2.211 for ID, K-OOD, N-OOD, and Joint OOD, respectively. All three corresponding training-seed comparisons favor LEAF in each group. The larger gains under team-size and joint extrapolation show the value of the relational encoder when a defender must coordinate within a larger entity set.

Figure 2 connects learning to deployment. At the tenth validation checkpoint, approximately 0.2 million environment steps, LEAF reaches -2.046 ± 0.028 compared with REFIL's -4.429 ± 0.284. At the final checkpoint, the rewards are

-0.373 ± 0.066 and -0.738 ± 0.122. The other panels follow the frozen policies as target count, team size, and both populations increase. Presenting each joint configuration separately preserves the distinct changes in threats and assets.

LEAF also has lower mean friendly-collision mortality than each learned baseline in all four groups. Rule incurs fewer collision deaths in *N*-OOD and Joint OOD, at 5.70% and 4.48%, compared with LEAF's 6.29% and 4.97%. Its corresponding rewards are lower. The two metrics therefore distinguish conserving defenders from protecting the targets: LEAF combines low friendly losses with more effective defense.

B. Ablation and Design Comparisons

The trained comparisons address three aspects of the encoder: constructing context before reading, locating temporal memory, and combining the relational feature with its host. Table III and Fig. 3 give both learning trajectories and population-transfer results. The *Memory query* design adds detached previous GRU state to the reader query. *Direct read*, *One round*, and *No host* use this same memory-query formulation, providing the paired design comparisons below.

Context before reading. Direct read applies the reader to the initial entity representations. One round performs one shared contextual update before reading. The latter improves group mean reward by 0.319, 0.755, 0.665, and 1.039 in ID, *K*-OOD, *N*-OOD, and Joint OOD, with two of three seed pairs improving in each group. The consistent direction across population groups supports processing entity relationships before compressing them into the defender's decision feature.

Current-state reading with downstream memory. Memory query and LEAF both preserve the recurrent decision network. Using only the current self state for the query gives LEAF mean reward gains of 0.122, 0.533, 0.424, and 0.849 across the four groups. The gains hold for three seeds in ID and two seeds in each extrapolation group. The result favors the chosen division of computation: the reader extracts the current relational context, and the downstream GRU combines it with temporal history.

Integration with cooperative value learning. No host retains the relational path while removing the REFIL feature path, imagined branches, and associated auxiliary loss. Restoring this host in the Memory query design improves the three extrapolation-group rewards by 0.239, 0.265, and 0.372; the ID mean changes by -0.017. LEAF then combines the retained host with current-state reading. Although No host reaches a final ID validation reward of -0.381 ± 0.126, close to LEAF's -0.373 ± 0.066, their transfer results differ substantially. This contrast shows why the evaluation must examine deployment populations alongside training validation.

A parameter-matched REFIL control has 176,393 actor parameters, close to LEAF's 175,754; standard REFIL has 102,025. Its group rewards are -0.914 ± 0.165, -2.493 ± 0.280, -2.269 ± 0.312, and -4.170 ± 0.760. LEAF exceeds this control in all three seed comparisons for every group. The performance gain thus persists when the host is given a comparable parameter budget.

C. Mechanism of Relational Encoding

The mechanism study follows the computation introduced in Section IV: contextual updates produce intermediate entity sets, the self-conditioned reader combines them into a decision feature, and the resulting actions determine interception and protection. We examine execution at the depths used during training, record the normal intermediate reads, and relate the policy's outcome to the formal protection metrics.

Protection across processing depths. We execute each frozen LEAF model at $L \in \{1,2,3,4\}$ on ten common scenes in each of three configurations: $(10,10,2)$, $(30,30,2)$, and $(30,30,12)$. The first two vary team size at fixed target count; the last two vary target count at fixed team size. All four paths use the trained update block, reader, fusion, and recurrent decision network. REFIL runs on the same scenes. Reward is averaged over scenes within each model and then reported across three model seeds; differences between depths are paired by model and scene.

Figure 4 shows that all four LEAF depths have higher mean reward than REFIL in these three configurations. At 10v10 and 30v30 with two targets, two-round execution has paired reward differences of -0.273 ± 0.047 and -0.363 ± 0.291 relative to four rounds, with all three model means lower. Three and four rounds give the same reward at 10v10; three rounds give 0.146 ± 0.131 higher reward at 30v30 with two targets and 0.793 ± 0.907 lower reward with twelve targets. Thus the ordering of execution depths changes with the engagement. Together with the trained Direct read–One round comparison, these results locate the common advantage in useful contextual representations, with later processing altering protection according to the scene.

Intermediate reading varies with the defender and time. We record all four read vectors and fusion weights during the normal four-round executions: 90 episodes containing 38,693 live-defender decisions. Figure 5(a) shows a clear preference for early context. The first read receives 73.66–75.65% of mean fusion weight across the three configurations, and the first two together receive approximately 99%. The encoder therefore supplies its recurrent policy primarily with the early contextual reads even when it executes all four updates.

The same average preference encompasses different choices by individual defenders. For the first-round weight, the within-state SD across active defenders averages 11.44, 13.91, and 12.68 percentage points in the three configurations. Tracking the same defender across adjacent physical steps gives mean absolute weight changes of 5.49, 5.34, and 5.36 percentage points. These measurements directly characterize the self-conditioned fusion: defenders observing the same engagement combine stages differently, and each defender changes that combination as the engagement evolves. The shared encoder consequently offers a common relational computation with receiver-specific access to its intermediate stages.

From coordinated response to target protection. The formal evaluation uses 300 scenes per model and configuration. Across the three mechanism configurations and the larger 50v50 case, LEAF combines higher attacker interception with lower friendly-collision mortality and less frequent target

TABLE II
FORMAL COMPARISON: MEAN ± SAMPLE SD. REWARD $R = -D$ IS HIGHER-IS-BETTER; FRIENDLY-COLLISION MORTALITY C IS LOWER-IS-BETTER. EACH LEARNED METHOD HAS THREE TRAINING SEEDS AND 300 COMMON SCENES PER CONFIGURATION. BOLD MARKS THE BEST GROUP MEAN FOR EACH METRIC.

Method	Metric	ID	K-OOD	N-OOD	Joint OOD
LEAF	$R\uparrow$	$\mathbf{-0.405 \pm 0.097}$	$\mathbf{-0.996 \pm 0.233}$	$\mathbf{-0.575 \pm 0.235}$	$\mathbf{-1.097 \pm 0.105}$
	$C\downarrow$ (%)	$\mathbf{1.33 \pm 0.45}$	$\mathbf{1.70 \pm 0.62}$	6.29 ± 1.32	4.97 ± 1.26
REFIL	$R\uparrow$	-0.715 ± 0.049	-1.796 ± 0.304	-2.209 ± 0.277	-3.308 ± 0.303
	$C\downarrow$ (%)	2.97 ± 0.42	4.71 ± 0.67	12.86 ± 2.14	10.34 ± 1.69
QMIX-Atten	$R\uparrow$	-2.318 ± 1.431	-5.009 ± 2.497	-5.139 ± 3.111	-7.415 ± 3.970
	$C\downarrow$ (%)	10.82 ± 9.56	12.42 ± 9.75	16.38 ± 5.43	15.37 ± 7.34
DCG	$R\uparrow$	-2.028 ± 0.437	-5.839 ± 1.920	-5.021 ± 2.658	-8.092 ± 3.909
	$C\downarrow$ (%)	9.05 ± 3.36	14.97 ± 4.68	19.70 ± 8.52	19.26 ± 10.20
SPECTra	$R\uparrow$	-2.662 ± 0.070	-6.933 ± 0.670	-6.455 ± 1.403	-11.184 ± 0.352
	$C\downarrow$ (%)	13.06 ± 0.75	20.71 ± 1.60	28.92 ± 1.54	31.25 ± 1.63
ALMA	$R\uparrow$	-1.326 ± 0.193	-3.022 ± 0.344	-3.605 ± 0.742	-5.375 ± 1.036
	$C\downarrow$ (%)	7.42 ± 1.13	9.05 ± 1.66	18.42 ± 3.84	15.92 ± 3.86
Rule†	$R\uparrow$	-1.462 ± 0.717	-3.212 ± 1.126	-1.545 ± 0.881	-3.060 ± 0.976
	$C\downarrow$ (%)	1.98 ± 4.47	2.70 ± 3.57	$\mathbf{5.70 \pm 3.30}$	$\mathbf{4.48 \pm 2.44}$

† Rule has no training seeds. Its SD is across the 300 scene seeds, after averaging configurations within each scene. All other SDs measure training-seed variation. Collision SDs are percentage points.



Fig. 2. Learning and population transfer. (a) Fifty validation checkpoints for LEAF and five learned baselines at recorded environment steps; bands show one model-seed sample SD. The Rule line marks its final ID reward. (b)–(c) Frozen-policy reward under target-count and team-size extrapolation, with seed points and SD bands for learned methods. (d) Reward at the eight joint-extrapolation configurations on a common linear scale.

damage than REFIL. At 10v10 with two targets, interception increases from 77.00% to 86.38%, and target-damage incidence falls from 55.22% to 30.44%. At 30v30 with two targets, interception rises from 75.11% to 83.02% and target-

TABLE III
TRAINED DESIGN COMPARISONS: ABSOLUTE REWARD $R = -D$ AND FRIENDLY-COLLISION MORTALITY C, MEAN ± SAMPLE SD ACROSS THREE TRAINING SEEDS. EACH CONFIGURATION USES 300 COMMON SCENES. BOLD MARKS THE BEST MEAN AMONG THESE FIVE DESIGNS.

Method	Metric	ID	K-OOD	N-OOD	Joint OOD
LEAF	$R\uparrow$	$\mathbf{-0.405 \pm 0.097}$	$\mathbf{-0.996 \pm 0.233}$	$\mathbf{-0.575 \pm 0.235}$	$\mathbf{-1.097 \pm 0.105}$
	$C\downarrow$ (%)	$\mathbf{1.33 \pm 0.45}$	$\mathbf{1.70 \pm 0.62}$	$\mathbf{6.29 \pm 1.32}$	$\mathbf{4.97 \pm 1.26}$
Memory query	$R\uparrow$	-0.527 ± 0.227	-1.529 ± 0.734	-0.998 ± 0.621	-1.946 ± 1.229
	$C\downarrow$ (%)	1.66 ± 0.86	2.59 ± 1.78	8.57 ± 4.41	6.65 ± 4.16
No host	$R\uparrow$	-0.510 ± 0.138	-1.768 ± 0.621	-1.263 ± 0.653	-2.319 ± 0.889
	$C\downarrow$ (%)	2.23 ± 0.77	3.67 ± 1.65	10.71 ± 3.98	8.76 ± 3.14
Direct read	$R\uparrow$	-0.928 ± 0.210	-1.958 ± 0.457	-2.020 ± 0.446	-3.196 ± 0.687
	$C\downarrow$ (%)	3.34 ± 1.66	4.46 ± 2.10	11.16 ± 1.81	9.17 ± 2.18
One round	$R\uparrow$	-0.609 ± 0.272	-1.203 ± 0.402	-1.355 ± 0.884	-2.157 ± 1.006
	$C\downarrow$ (%)	1.57 ± 0.15	2.08 ± 0.32	8.35 ± 3.16	6.56 ± 2.37

Configurations are weighted equally within each training seed. Memory query adds incoming GRU history to LEAF's reader query. No host removes the local host, imagination, and associated losses from the memory-query design. Direct read and One round retain query history and execute zero and one contextual update, respectively. Collision SDs are percentage points.

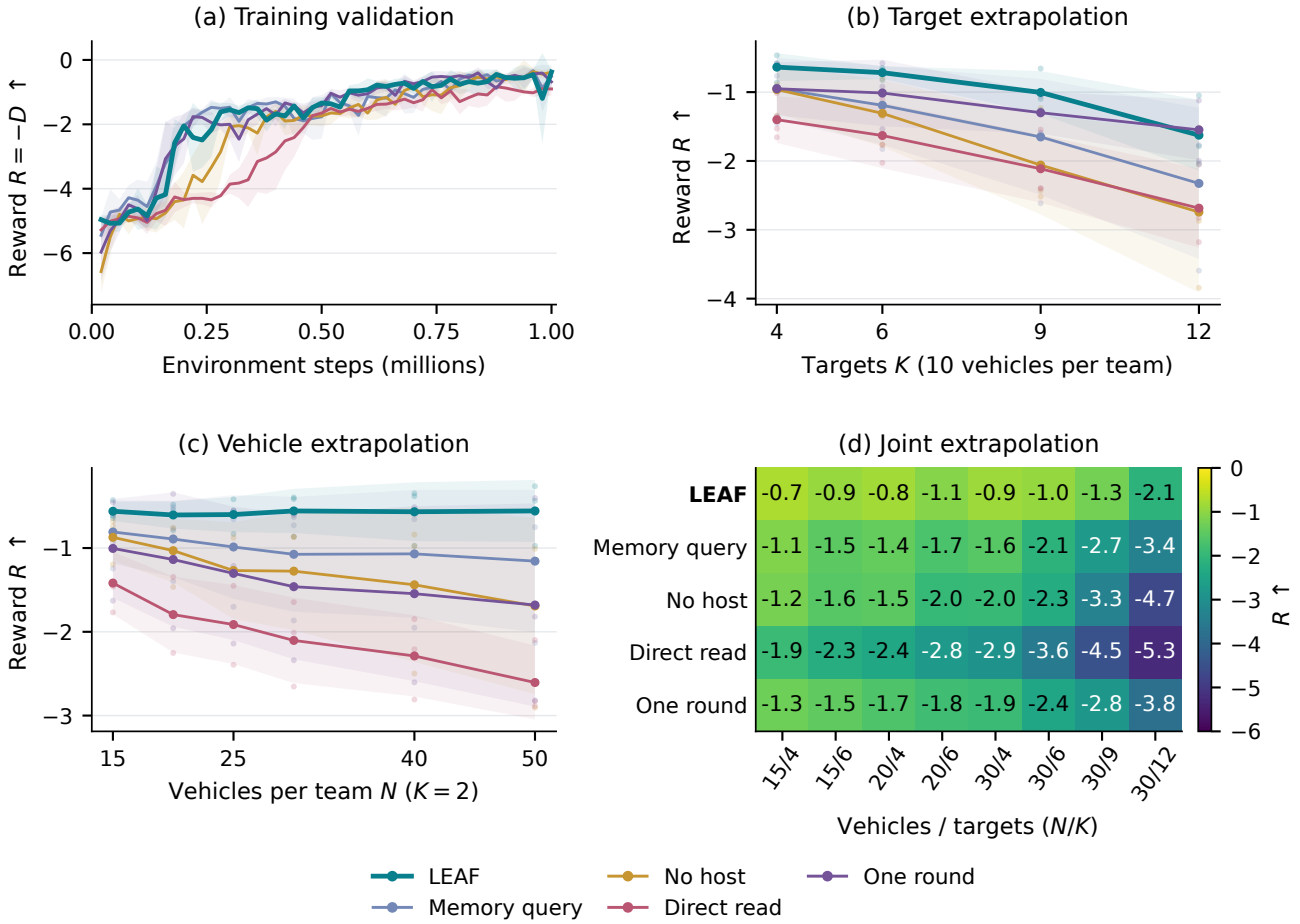


Fig. 3. Trained design comparisons. (a) Validation reward over 50 checkpoints; lines and bands show the three-model mean and sample SD at actual mean environment steps. (b)–(c) Target-count and team-size extrapolation. (d) Joint-extrapolation rewards on a common linear scale. Memory query adds temporal state to LEAF's reader. Direct read, One round, and No host use that memory-query formulation; their paired comparisons are described in the text.

damage incidence falls from 70.89% to 29.56%. Increasing the targets from two to twelve at the same team size preserves this direction: LEAF intercepts 81.54% of attackers and experiences target damage in 48.33% of episodes, compared with 74.48% and 78.22% for REFIL. At 50v50, LEAF retains an interception advantage of 6.41 percentage points and reduces friendly-collision mortality from 19.59% to 11.14%; target-damage incidence falls from 70.56% to 28.11%. The learned representation thus supports the intended team outcome: fewer defenders are lost to one another, more threats are intercepted,

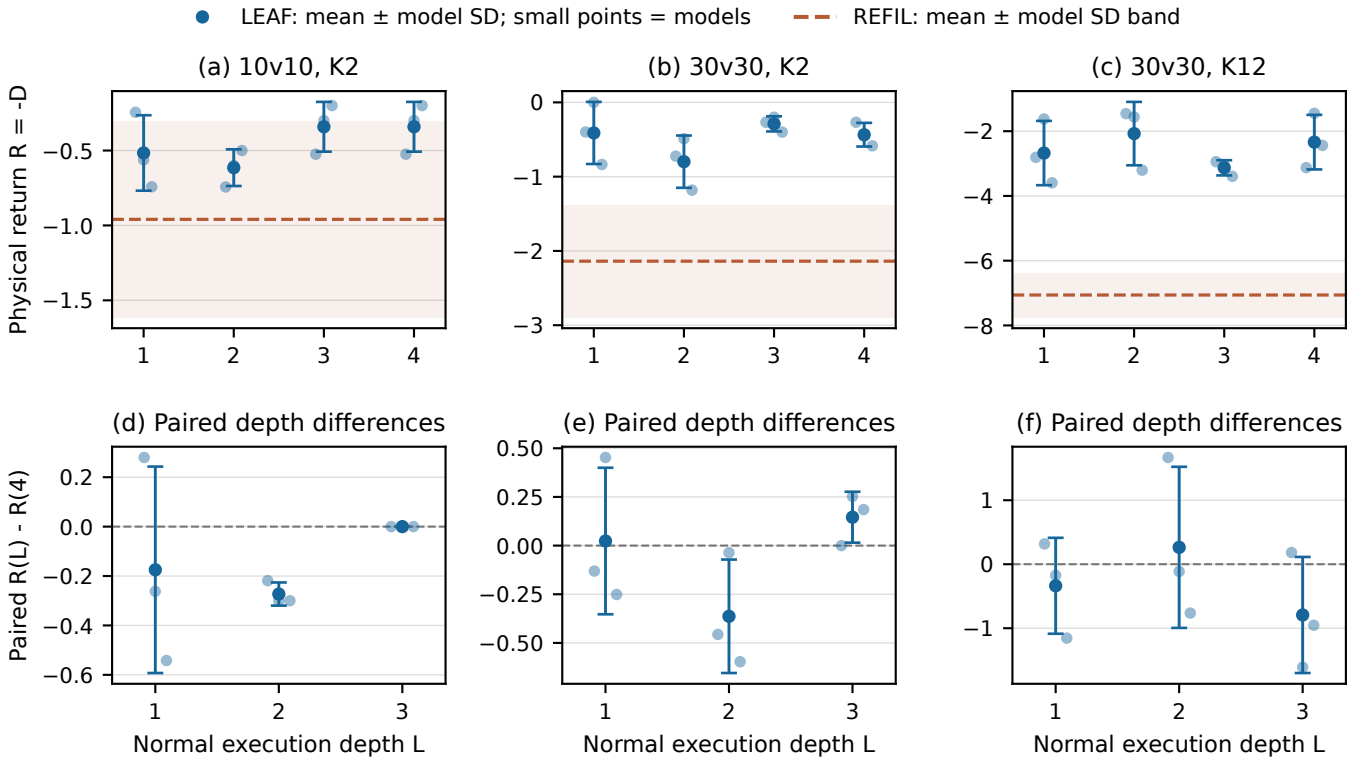


Fig. 4. Frozen LEAF execution at depths used during training. Top: reward at each depth and the REFIL reference. Bottom: scene-paired reward difference from four-round execution; positive values favor the shorter path. Each point and error bar is the mean and sample SD of three model means, each based on ten scenes. Small points show individual models.

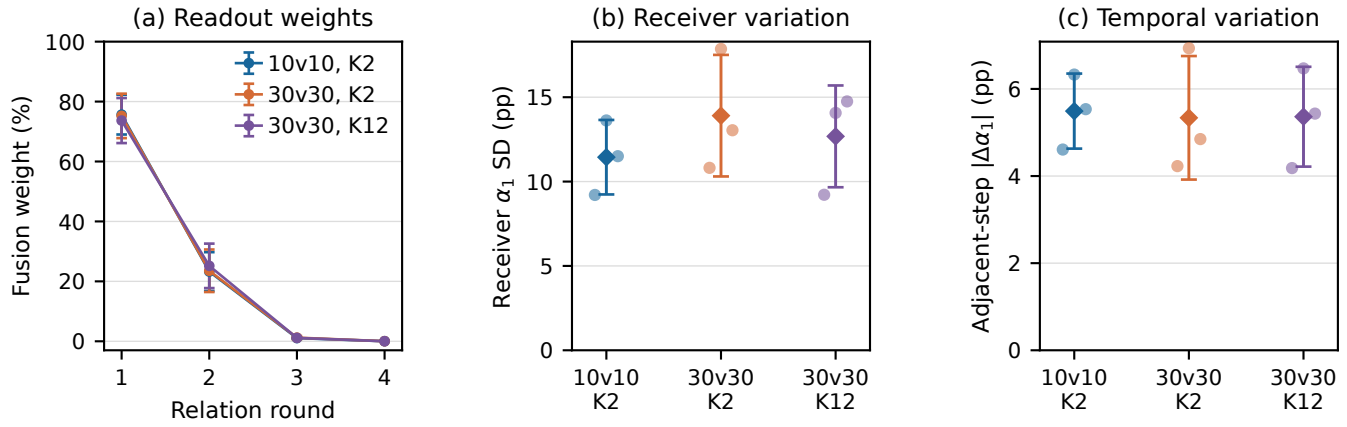


Fig. 5. Intermediate-read use in normal four-round execution. (a) Mean fusion weights by round and configuration. (b) Variation among active defenders at the same physical step, measured by the population SD of the first-round weight. (c) Mean absolute change of that weight for the same defender at adjacent physical steps. Panel (b) averages over physical decision states within each model. Panel (c) averages the adjacent-step changes within each episode and then weights the ten scenes equally. Error bars show sample SD across three model means; small points retain the models in (b)–(c).

and fewer episodes allow damage to the protected assets.

D. Representative Trajectories

Figure 6 compares all seven methods at $(5,5,2)$ and $(15,15,2)$ under the same initial layout within each configuration. Learned policies use training seed 0, and both examples use scene seed 9000. Paths and entity states are shown at physical step 30; the adjacent reward curves continue to each policy's actual terminal step.

At 5v5, LEAF, REFIL, and ALMA finish with reward -1. LEAF and REFIL first incur target damage at step 20 and complete their engagements at steps 32 and 31. Their trajectories show different interception routes with the same final protection outcome.

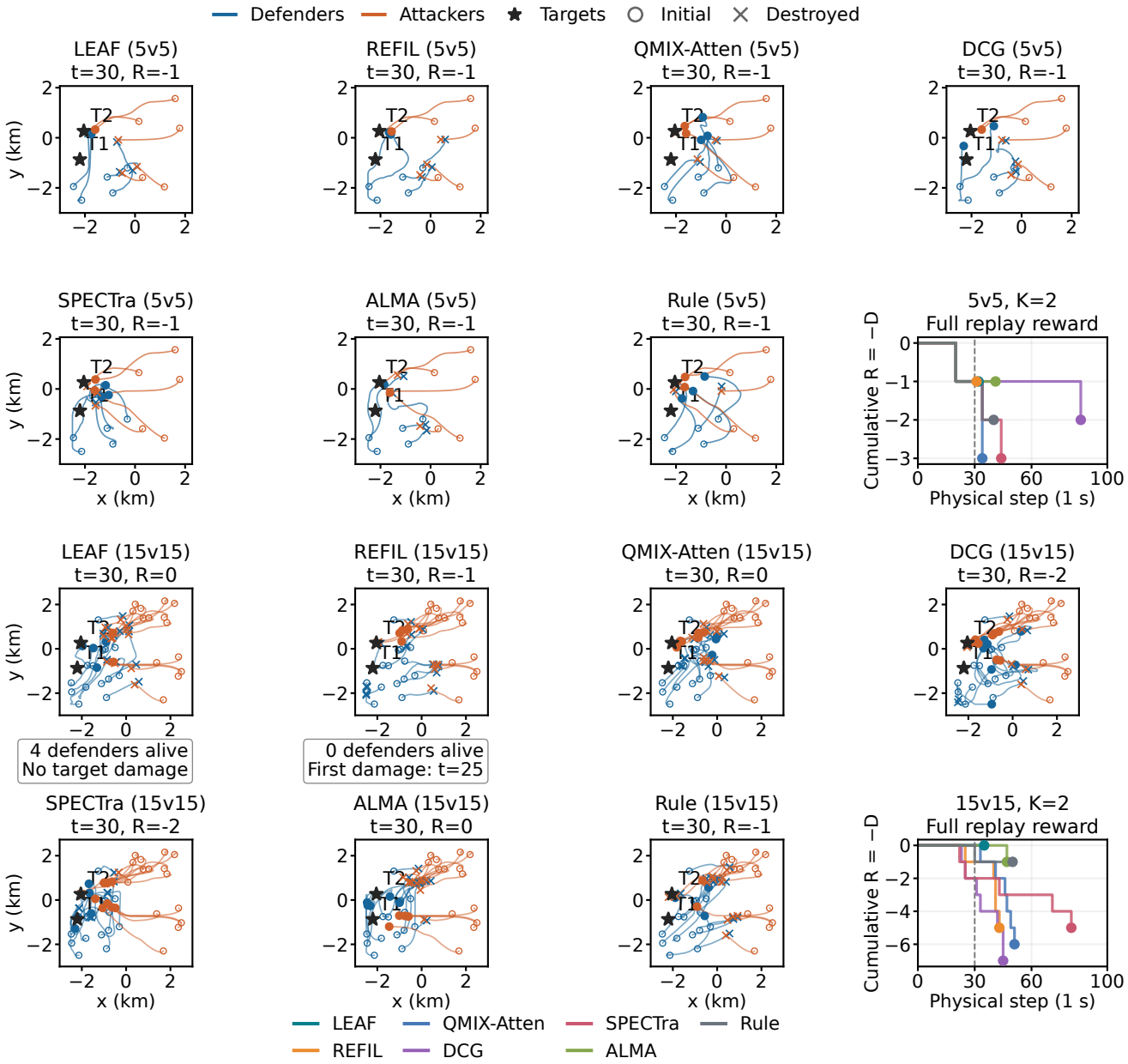


Fig. 6. Common-scene trajectory comparison. The upper two rows show 5v5 and the lower two show 15v15, both with two targets. Each block contains seven spatial panels and a cumulative-reward plot. Paths end at the common snapshot step 30: open circles are initial positions, filled circles are live vehicles, crosses are destroyed vehicles, and stars are targets. Panel titles give reward at the snapshot. Reward curves end at the actual terminal step, marked by a dot; the dashed line marks the snapshot time. The annotations identify the two methods' defensive state at step 30 in the 15v15 case. All spatial panels within a scale share coordinates and color conventions.

The 15v15 scene exposes the effect of sustaining the defense. At the common snapshot, LEAF retains four defenders against two surviving attackers, with both targets undamaged. REFIL has lost its final defender at step 27; four attackers remain at step 30, and reward has reached -1. Its reward then falls to -5 as damage continues after the defending team disappears. LEAF intercepts all 15 attackers by step 35 and finishes with $R = 0$. QMIX-Atten, DCG, SPECTra, ALMA, and Rule finish with -6, -7, -5, -1, and -1, respectively. The surviving-team snapshots and complete reward histories connect maintained defensive capability to the final protection outcome.

VI. Conclusion

LEAF uses a shared loop and self-conditioned intermediate reading to build a relational feature for cooperative target protection. The encoder develops the current entity context, combines reads from successive processing stages, and supplies the result to a recurrent utility network. This organization supports a common policy across changing numbers of defenders, attackers, and targets.

In SkyHAD, frozen LEAF policies attain higher group mean reward than five learned baselines and Rule across the four evaluated population groups. Trained comparisons

favor constructing context before reading and using current-state queries with temporal memory downstream. The gains persist against a parameter-matched REFIL control. At larger populations, higher attacker interception and lower target-damage incidence connect the representation design to its intended protective outcome. The resulting controller provides an effective simulation-based approach to transferring cooperative defense policies from small training engagements to larger deployment populations.

ACKNOWLEDGMENT

OpenAI Codex assisted with manuscript drafting, code-assisted analysis, and figure preparation.

REFERENCES

[1] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Poczos, R. Salakhutdinov, and A. J. Smola, "Deep sets," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 3391–3401.

[2] J. Lee, Y. Lee, J. Kim, A. Kosiorek, S. Choi, and Y. W. Teh, "Set transformer: A framework for attention-based permutation-invariant neural networks," in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 97, 2019, pp. 3744–3753.

[3] S. Iqbal, C. A. S. de Witt, B. Peng, W. Boehmer, S. Whiteson, and F. Sha, "Randomized entity-wise factorization for multi-agent reinforcement learning," in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 139, 2021, pp. 4596–4606.

[4] "SkyHAD," Software, version 3.0.0, 2026, [Online]. Available: https://github.com/Sailero/SkyHAD.

[5] E. Garcia, D. W. Casbeer, and M. Pachter, "Design and analysis of state-feedback optimal strategies for the differential game of active defense," *IEEE Transactions on Automatic Control*, vol. 64, no. 2, pp. 553–568, 2019.

[6] M. Chen, Z. Zhou, and C. J. Tomlin, "Multiplayer reach-avoid games via pairwise outcomes," *IEEE Transactions on Automatic Control*, vol. 62, no. 3, pp. 1451–1457, 2017.

[7] R. Yan, Z. Shi, and Y. Zhong, "Task assignment for multiplayer reach–avoid games in convex domains via analytical barriers," *IEEE Transactions on Robotics*, vol. 36, no. 1, pp. 107–124, 2020.

[8] R. Yan, X. Duan, Z. Shi, Y. Zhong, and F. Bullo, "Matching-based capture strategies for 3D heterogeneous multiplayer reach-avoid differential games," *Automatica*, vol. 140, p. 110207, 2022.

[9] D. Shishika, J. Paulos, and V. Kumar, "Cooperative team strategies for multi-player perimeter-defense games," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 2738–2745, 2020.

[10] B. P. Gerkey and M. J. Matarić, "A formal analysis and taxonomy of task allocation in multi-robot systems," *The International Journal of Robotics Research*, vol. 23, no. 9, pp. 939–954, 2004.

[11] H.-L. Choi, L. Brunet, and J. P. How, "Consensus-based decentralized auctions for robust task allocation," *IEEE Transactions on Robotics*, vol. 25, no. 4, pp. 912–926, 2009.

[12] R. Deng, R. Yan, P. Huang, Z. Shi, and Y. Zhong, "A distributed auction algorithm for task assignment with robot coalitions," *IEEE Transactions on Robotics*, vol. 40, pp. 4787–4804, 2024.

[13] A. Miele, M. Lippi, and A. Gasparri, "A distributed framework for integrated task allocation and safe coordination in networked multi-robot systems," *IEEE Transactions on Automation Science and Engineering*, vol. 22, pp. 11 219–11 238, 2025.

[14] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[15] C. Yu, A. Velu, E. Vinitsky *et al.*, "The surprising effectiveness of PPO in cooperative multi-agent games," in *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, vol. 35, 2022.

[16] B. Liu, Q. Liu, P. Stone, A. Garg, Y. Zhu, and A. Anandkumar, "Coach-player multi-agent reinforcement learning for dynamic team composition," in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 139, 2021, pp. 6860–6870.

[17] Z. Peng, Z. Lu, X. Mao *et al.*, "Multi-ship dynamic weapon-target assignment via cooperative distributional reinforcement learning with dynamic reward," *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 9, no. 2, pp. 1843–1859, 2025.

[18] Q. Wu, K. Liu, L. Chen, and J. Lü, "Hierarchical reinforcement learning for swarm confrontation with high uncertainty," *IEEE Transactions on Automation Science and Engineering*, vol. 22, pp. 8630–8644, 2025.

[19] Y. Liu, C. Liu, Y. Meng, B. Jiang, and X. Wang, "Observer-based multi-agent reinforcement learning for pursuit-evasion game with multiple unknown uncertainties," *IEEE Transactions on Automation Science and Engineering*, vol. 22, pp. 14 332–14 345, 2025.

[20] X. Li, Z. Li, X. Yang, X. Yu, and J. Qiu, "Multi-agent target defense differential game: A hierarchical recognition-allocation-execution learning approach," *International Journal of Robust and Nonlinear Control*, vol. 35, no. 16, pp. 6868–6879, 2025.

[21] M. Hüttenrauch, A. Šošić, and G. Neumann, "Deep reinforcement learning for swarm systems," *Journal of Machine Learning Research*, vol. 20, no. 54, pp. 1–31, 2019.

[22] P. Sunehag, G. Lever, A. Gruslys *et al.*, "Value-decomposition networks for cooperative multi-agent learning based on team reward," in *Proceedings of the 17th International Conference on Autonomous Agents and Multiagent Systems*, 2018, pp. 2085–2087.

[23] T. Rashid, M. Samvelyan, C. Schroeder, G. Farquhar, J. Foerster, and S. Whiteson, "QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 80, 2018, pp. 4295–4304.

[24] W. Boehmer, V. Kurin, and S. Whiteson, "Deep coordination graphs," in *Proceedings of the 37th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 119, 2020, pp. 980–991.

[25] S. Hu, F. Zhu, X. Chang, and X. Liang, "UPDeT: Universal multi-agent reinforcement learning via policy decoupling with transformers," in *International Conference on Learning Representations*, 2021.

[26] M. Gallici, M. Martin, and I. Masmitja, "TransfQMix: Transformers for leveraging the graph structure of multi-agent reinforcement learning problems," in *Proceedings of the 22nd International Conference on Autonomous Agents and Multiagent Systems*, 2023, pp. 1679–1687.

[27] H. Park, B. Seong, and S.-K. Ko, "Scalable multi-agent reinforcement learning with permutation-free networks," *IEEE Access*, vol. 14, pp. 95 650–95 664, 2026.

[28] S. Iqbal, R. Costales, and F. Sha, "ALMA: Hierarchical learning for composite multi-agent tasks," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 7155–7166.

[29] H. Tan, S. Zhang, S. Lin, and B. Huang, "GR-MAPPO algorithm for perimeter defense problem in multi-agent systems," *Entropy*, vol. 28, no. 6, p. 659, 2026.

[30] V. Zambaldi, D. Raposo, A. Santoro *et al.*, "Deep reinforcement learning with relational inductive biases," in *International Conference on Learning Representations*, 2019.

[31] M. Dehghani, S. Gouws, O. Vinyals, J. Uszkoreit, and L. Kaiser, "Universal transformers," in *International Conference on Learning Representations*, 2019.

[32] K. Xu, C. Li, Y. Tian, T. Sonobe, K. ichi Kawarabayashi, and S. Jegelka, "Representation learning on graphs with jumping knowledge networks," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 80, 2018, pp. 5453–5462.

[33] A. Fan, E. Grave, and A. Joulin, "Reducing transformer depth on demand with structured dropout," in *International Conference on Learning Representations*, 2020.

[34] T. Phan, F. Ritz, P. Altmann *et al.*, "Attention-based recurrence for multi-agent reinforcement learning under stochastic partial observability," in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 27 840–27 853.

[35] R. Agarwal, M. Schwarzer, P. S. Castro, A. C. Courville, and M. G. Bellemare, "Deep reinforcement learning at the edge of the statistical precipice," in *Advances in Neural Information Processing Systems*, vol. 34, 2021.